

DATA266 Lab1 Fall 2026

LLM Pretraining · Sentiment Classification · CycleGAN Style Transfer

Objective: This lab simulates how AI teams work in industry — individuals build and experiment independently, then the team synthesizes results, agrees on shared evaluation, and ships one collaborative report. Use it to learn both the technical work and the teamwork.

All resource links (quick reference): [TinyStories dataset](#) | [TinyStories paper](#) | [Attention Is All You Need](#) | [yelp polarity](#) | [NLP Sentiment Classification](#) | [Kaggle competition signup](#) | [CycleGAN paper](#)

Before you start: confirm your exact deadline and team size/grouping on Canvas — this document covers what to do and how it's graded.

1. General Submission Instructions (Team)

- Create one shared GitHub repository for the team. Every member commits under their own named folder inside each task folder (task1_llm/, task2_sentiment/, task3_gan/).
- The repo must include a top-level README.md explaining setup, how to reproduce any member's run, and where each result lives.
- No personal file paths, credentials, or API keys in any committed file.
- Keep raw training logs unedited after each run — they are your evidence trail for grading.
- The team submits one combined final report (see Section 9) referencing every member's individual work
- **Book GPU Lab time well in advance — don't wait until the last week.** Slots are limited and shared across the whole class; if you wait too long, you may not get a slot in time to actually train your model before the deadline.
- **Back up your work as you go, not just at the end.** GPU Lab machines are often wiped clean after your assigned session ends — anything left only on the lab machine (checkpoints, logs, code) can be lost permanently with no way to recover it. Get in the habit of copying your work off the lab machine regularly (to your GitHub repo or personal storage), rather than treating backup as a final step you'll do once training is done.
- To reserve lab machines, please use the following reservation form:

https://docs.google.com/forms/d/1liixZWdnZNbL402KZp4hNpnWCnpRTXF4xrdq_Ne61A/viewform?edit_requested=true&pli=1&fbzx=-8062309470645377095

2. What's Expected From Each Individual

- Each member independently designs, codes, and trains their own model for all three tasks — own architecture, own hyperparameters, own results. Two members cannot submit near-identical models.
- Each member documents their own work in a results.md inside their folder: what you built, why, and your metrics.
- Each member is individually responsible for their own metrics table, failure/error analysis, and hardware disclosure.

- Each member must be able to defend their own model choices and results if asked.

3. Scoring — 50% Team, 50% Individual

Score Type	Weight	What Is Evaluated
Individual	50%	Your own model builds across all 3 tasks, your own metrics tables, your own error/failure analysis, your own hardware disclosure, and your Demo/Viva defense.
Team	50%	The shared repo quality, the combined comparison tables across all members' models, the joint written synthesis/analysis in the final report, and reproducibility of the whole team's work.

Marks by Component (100 total, +10 bonus)

Component	Individual	Team	Total
Task 1 — LLM from Scratch	15	15	30
Task 2 — Sentiment Classification	15	15	30
Task 3 — CycleGAN (+ Kaggle leaderboard bonus)	15	15	30 (+10)
Reproducibility & Professional Practice	0	5	5
Demo/Viva — Individual scored	5	0	5
Total	50	50	100 (+10)

Reproducibility is scored purely at the team/repo level (Section 8); the Demo/Viva is scored purely at the individual level (Section 4) — see each section for exactly what's graded.

Both scores are mandatory — a strong individual model with a weak team report (or vice versa) will not earn full marks.

4. Demo / Viva : (5 Marks)

The **team presents and demos together** in one shared session — but within it, each **member is scored individually** on their own contribution. Showing up prepared for your own portion is on you; it isn't averaged across the team.

Format

- The team walks through the demo together, but each member presents and defends their own models across all 3 tasks.

What to bring

- Your own trained models/checkpoints, ready to run or demo live if asked.

- Your own results.md and metrics tables for all 3 tasks, open and ready to reference.

What you'll be asked to explain(any or all the below)

- Your architecture and hyperparameter choices for each task, and why you made them.
- How your reported metrics were computed.
- Your failure/error analysis findings (Task 1 failure cases, Task 2 error review).
- How your model differs from your teammates' models and what that comparison showed.

Graded on (per individual, not per team)

- Depth of understanding of your own implementation — not just what it does, but why.
- Ability to justify design decisions and answer follow-up or "what if" questions.
- Clarity of explanation.

Your leaderboard rank or benchmark score is not what's being judged here — your understanding of your own work is. A confident teammate cannot carry you: if you can't explain your own model, that reflects in your own score only.

5. Reproducibility & Professional Practice (5 marks — Team)

This is graded once for the whole team, based on the shared repo. Two things are expected of every member individually, but they support the team score rather than being marked separately:

- Keep an unedited raw log file for every training run — this is your evidence trail; do not clean it up after the fact.
- Record a manifest for your own work: package/library versions, environment file, and which checkpoint maps to which result in your report.

Graded criteria (5 marks)

- The README must let a grader clone the repo and reproduce a smoke test (any one member's run) using a single documented command.
- No hard-coded personal file paths or secrets anywhere in the repo; config-driven runs only.
- Every member's raw logs and manifests are present and untouched since the run.

TypeScript

team-repo/

|— README.md

|— task1_llm/

| |— data/ ← shared raw TinyStories dataset

| |— member_A_name/

| |— member_B_name/

```

|—— task2_sentiment/
|   |—— data/                ← shared raw IMDB dataset
|   |—— member_A_name/
|   |—— member_B_name/
|—— task3_gan/
|   |—— data/                ← shared Monet/Photo images
|   |—— member_A_name/
|   |—— member_B_name/
|—— reproducibility/
|   |—— manifests/
|   |—— raw_logs/
|—— report/
    |—— DATA266_Lab1_Report_Team_[Team Number].pdf

```

For each member under each task :

HTML

```

member_X_name/
|—— src/                    ← your code (task-specific ipynb files inside with output)
|—— data_processed/        ← YOUR OWN preprocessing output – never shared (if applicable)
|—— checkpoints/           ← your trained model weights
|—— outputs/               ← generated results (samples, predictions, plots, confusion matrices
- subfolders as needed)
|—— metrics_report.csv      ← every required metric for this task, in one file
|—— failure_analysis.md     ← required failure/error case write-up for this task
|—— results.md             ← architecture + hyperparameter justification, tying it all together

```

6. Final Report Submission — Instructions

1. Submit one combined PDF report at report/DATA266_Lab1_Report_Team_[Team Number].pdf in the shared repo. - also include the repo link in your report.
2. Open with a one-paragraph team ownership statement: who built what.
3. For each of the 3 tasks, include one comparison table listing every member's model, its architecture summary, its hyperparameters, and all metrics from that task's list above, side by side.
4. For each task, add a short jointly written analysis: strengths, weaknesses, limitations, and what the team would try next.

5. Attach evidence for every number: link loss-curve images, logs, sample generations, and checkpoint IDs so metrics are traceable to their source.
 6. Include each member's failure/error analysis (Task 1 failure cases, Task 2 error review) with the actual text/image snippets attached.
 7. Cite the papers referenced in the brief (Attention Is All You Need, TinyStories, CycleGAN).
 8. Push the report to the repo before the deadline.
-

7. Pre-Submission Checklist

- Every member's folder exists under all 3 task folders with code, config, logs, and results.md
 - No two members' models (within the team) share the same architecture + hyperparameters
 - Task 1: no prebuilt Transformer/attention modules used
 - Task 2: no pretrained embeddings or pretrained LMs used; all 3 models trained per member
 - All required metrics reported for every model, every member (Tasks 1–3)
 - Task 3 Kaggle submission made and leaderboard rank recorded
 - Comparison tables (architecture + hyperparameters + metrics) present in the report for all 3 tasks
 - Raw logs and manifests committed and untouched since the run
 - README allows one-command reproduction of at least one member's run
 - No hard-coded personal paths or secrets anywhere in the repo
 - Final combined PDF report pushed to report/
 - Every member is ready for their individual Demo/Viva (Section 4) — checkpoints runnable, results.md open, ready to justify choices
-

8. Best Practice Tips

- Commit early and often, with clear messages — your commit history is part of the evidence the team is graded on.
- Agree as a team on one evaluation script per task before anyone starts training, so every member's metrics are directly comparable.
- Use a shared config file (YAML/JSON) per model instead of hard-coded values — it makes your run reproducible and easy to compare.
- Keep a running experiment log (what you tried, what changed, what happened) — it turns into your failure/error analysis almost for free.
- You **CAN'T** use Claude (or a similar AI assistant) - for the core architecture decisions and analysis must be your own understanding, since you'll be asked to defend them.
- Drafts, review and test everything yourself before committing — you are accountable for what's pushed to the repo.

- Sync with your team regularly, not just at the end — the team score depends on catching mismatched protocols or formats early, not fixing them the night before submission.
- Back up *checkpoints and logs outside your local machine* (repo, cloud storage) — **lost training runs cannot be re-created before the deadline.**

Task 1 Build a GPT-Style LLM From Scratch

Your Role: AI Research Engineer, Language Modeling Team

You've been asked to build a from-scratch reference GPT implementation on the TinyStories dataset — the same exercise used to validate that engineers understand attention, causal masking, and autoregressive generation before touching production-scale models. No prebuilt Transformer or attention modules allowed.

Resources: [TinyStories dataset](#) | [TinyStories paper](#) | [Attention Is All You Need paper](#)

Instructions

1.1 Data Preprocessing (5 Marks)

1. Load the dataset and tokenize the text at the character level.
2. Create fixed-length input–target sequences suitable for autoregressive language modelling.
3. Convert characters into integer encodings using your own `char_to_idx` and `idx_to_char` dictionaries.
4. Split the dataset into training (100K) and validation (10K) sets — each member creates their own split.

1.2 GPT Model Implementation (15 Marks)

1. Implement one or more Transformer blocks from scratch, each consisting of: Multi-Head Self-Attention, Layer Normalization, Feed-Forward Network, and Residual connections.
2. Use causal masking in self-attention so the model cannot access future tokens during training.
3. Create learnable embedding layers for token embeddings and positional embeddings.
4. Add a language modelling head that projects the final hidden representations to the vocabulary size for next-token prediction.

1.3 Training and Text Generation (8 Marks)

1. Train the model using cross-entropy loss.
2. Implement learning rate warm-up and scheduling.
3. Train the model for a minimum of 10 epochs.
4. Plot and report training and validation loss curves.
5. Generate text samples using greedy decoding or temperature-based sampling.

1.4 Sequence Model Failure Analysis (2 Marks)

1. Analyse three failure cases observed in your generated text.
2. For each case, provide the generated text snippet and identify the type of failure (e.g., repetition, broken grammar, loss of coherence, hallucination) along with your observation.

Task 1 — Evaluation Metrics (report all)

- Training cross-entropy loss
- Validation cross-entropy loss
- Perplexity
- Bits-per-character
- Generalization gap (train vs. val loss)
- Top-1 next-character accuracy
- Distinct-1/2/3 (generation diversity)
- Repeated 4-gram rate
- Gradient norm / training stability (loss spikes, NaNs)
- Parameter count
- Training tokens/sec and generation tokens/sec
- Peak memory usage and total training time

*Use of prebuilt Transformer or attention modules is **not permitted**.*

Task 2 Yelp Polarity Sentiment Classification

Your Role: Applied NLP Researcher, Text Classification Team

Your team is running a model bake-off for binary sentiment classification with no pretrained embeddings or pretrained language models allowed — the same constraint applied ML teams face in low-resource or regulated settings. You will build and defend your own baseline-to-experimental model lineup.

Resources: [yelp polarity](#) | [NLP Sentiment Classification](#)

Instructions

2.1 Data Preprocessing (10 Marks)

1. Analyse review length distribution, class distribution and class balance.
2. Handle missing values or malformed entries, if any.
3. Perform text preprocessing, including but not limited to:
 - lowercasing
 - removal of punctuation and special characters
 - stopword removal
 - stemming or lemmatization (if applicable)
4. Tokenise the processed text.
5. Create textual embeddings learned from scratch.

2.2 Model Training and Evaluation (15 Marks)

1. Train 3 of your own models: a Baseline, and two Experimental models that each change architecture or hyperparameters — your set of 3 must differ from your own other models and from every other member's models.
2. Justify the model architecture and embedding choices for each model.
3. Evaluate each model using the full metrics list below.
4. Manually review 20 of your own model's errors: 5 confident false positives, 5 confident false negatives, 5 near-threshold errors, 5 slice-specific failures — assign an error type to each and propose one testable fix.
5. Record the exact CPU/GPU used for training each model.

2.3 Comparative Analysis and Observations (5 Marks)

1. Compare your own experimental models using the metrics list below, then compare against your teammates' models as a team.
2. Discuss observed strengths, weaknesses, and limitations.
3. Briefly comment on possible improvements or future work.

Task 2 — Evaluation Metrics (report all, per model) - *Recommend to learn the interpretation of the Metrics*

- Accuracy
- Precision, Recall, F1-score (macro, micro, and weighted)
- Confusion matrix
- ROC-AUC
- PR-AUC
- Matthews correlation coefficient (MCC)
- Brier score (calibration)
- Expected calibration error
- 95% bootstrap confidence intervals for accuracy, macro-F1, and MCC
- Paired McNemar test (baseline vs. each experimental model)
- Macro-F1 and error rate per data slice (robustness)
- Parameter count, training time, and examples/sec
- Peak memory usage

Task 3 CycleGAN Image Style Transfer

Your Role: Generative Modeling Researcher, Computer Vision Team

Your team is entering a shared Kaggle competition to build unpaired image-to-image translation — the technique behind production style-transfer and domain-adaptation pipelines. Each member trains their own CycleGAN; the team's leaderboard standing and analysis depend on comparing results and diagnosing issues together.

Resources: [Kaggle Class Competition signup](#) | [CycleGAN paper](#)

Instructions

3.1 Model Implementation and Training (20 Marks)

1. Prepare two unpaired image domains from the dataset.
2. Implement a CycleGAN architecture consisting of two generators and two discriminators.
3. Train the model using adversarial loss and cycle-consistency loss.
4. Perform image translation between the two domains.
5. Analyse training behaviour, including convergence and stability.

3.2 Evaluation and Analysis (10 Marks)

1. Assess the visual quality of your translated images.
2. Verify correct implementation of the cycle-consistency constraints.
3. Analyse training stability and loss behaviour.
4. Discuss the quality of your analysis, observations, and any shortcomings.
5. Submit predictions to the Kaggle Class Competition under your team and record your leaderboard rank.
6. Participate in a blinded human audit of 30 fixed samples (style, content, artifacts) with 2 raters, and report inter-rater agreement.

Leaderboard integrity — read before submitting to Kaggle:

Your submission must be the direct output of your own trained CycleGAN — no manually edited, hand-picked, copied, or externally-sourced images, no hardcoded or lookup-table outputs, no training on or peeking at test-set pairings, and no pretrained/foundation image models used to generate or touch the submitted images. Leaderboard rank earns marks only as evidence of a working model — a high rank achieved by any means other than your own CycleGAN's inference will be treated as an academic integrity violation and scored as zero for Task 3, regardless of reported metrics.

Task 3 — Evaluation Metrics (report all, both directions)

- **FID (Fréchet Inception Distance), both directions**
- **KID (Kernel Inception Distance), both directions**
- Generative precision and recall (or density/coverage)
- Cycle-reconstruction L1 distance
- LPIPS (perceptual similarity)
- Content-preservation cosine similarity (input vs. translation)
- Generator and discriminator loss curves
- Cycle-consistency and identity loss values
- Gradient norms / NaN count (training stability)
- Human audit score and inter-rater agreement (Cohen's kappa or % agreement)
- Parameter count, training time, images/sec, and peak memory
- Kaggle public/private leaderboard score (must come from your own trained model's inference — see integrity note above)

Leaderboard Rank	Points Awarded
1 - 10	10
11 - 20	9
21 - 30	8
31 - 40	7
40+	6

Git Repo Structure:

TypeScript

task3_gan/

|— **data/**

| |— **monet_jpg/** ← shared real Monet images

| |— **photo_jpg/** ← shared real Photo images

|— **member_A_name/**

| |— **src/**

| | |— **code** ipynb file

| | |— **full metrics file** ← KID, precision/recall, LPIPS, etc.

| |— **checkpoints/**

| |— **outputs/**

| | |— **pred_A2B/**

| | |— **pred_B2A/**

| |— **evaluate_local.py**

| |— **submission.csv** ← generated (Kaggle: FID/MiFID only)

| └── full_metrics_report.csv ← generated (everything **else**, **for** the report)

└── member_B_name/

 └── (identical structure, independent content)
